Title: Comparison of Thread Counting Using Static and Non-Static Variable:

Objective: By compiling this code:

— Understand the difference between static and non-static variables.

— Create and execute multiple threads.

— Observe shared and instance-level data in multithreaded program.

— Calculate the difference

— Percentage the difference between two count.

— Analyze the thread-effect on result.

— Understand the race conditions and thread safety.

Code:

```
import java.util.concurrent.atomic.AtomicLong;
public class Iffat_Thread{
    Static AtomicLong safe = new AtomicLong(0)
    Static long unsafe = 0;
    Static class Counter implements Runnable{
        long count=0;
        int n;
        boolean mode;
        Counter (int n, boolean mode){
```

```
        this.n=n;
        this.mode=mode;
    }
    public void run(){
        for(int i=0;i<n;i++){
            if (mode)
                Safe.incrementAndGet();
            else
                Unsafe++;
            count++;
        }
    }
}
public static void main(String[]args)
        throws Interruped Exception{
    int t = Integer.parseInt(args[0]);
    int n = Integer.parseInt(args[1]);
    boolean mode = Boolean.parseBoolean(args[2]);

    Safe.set(0);
    Unsafe = 0;
    Thread[]th=new Thread[t];
    Counter[]c = new Counter[t];

    for(int i=0;i<t;i++){
        c[i]=new Counter(n,mode);
        th[i] = new Thread(c[i]);
        th[i]=Start();
    }
    long total=0;
```

IT-24035

```
for(int i=0;i<t;i++){
  th[i].join();
  total+=c[i].count;
}
long s=mode?safe.get():unsafe;
long diff=Math.abs(s-total);
double percent=total==0
        ?(s==0?0:Double.NaN)
        :diff*100.0/total;
  System.out.println("Thread:"+t);
  System.out.println("Expected:"+(long)t*n);
  System.out.println("Static:"+s);
  System.out.println("Non-static total:"+total);
  System.out.println("Abosolute difference:"+diff);
  System.out.println("percentage difference:%.2f%%%n
      percent);
  }
}
```

JT-24035

Outpat:

Test	Threads	Increments/Threads	Expected count	Non-Static total
1	1	1000	1000	1000
2	2	10,000	20000	20,000
3	5	10,000	50000	50,000
4	10	50,000	500000	500000
5	20	50,000	1000000	1000000
6	50	50,000	2500000	2500000
7	100	50,000	5000000	5000000

Conclusion:

Static vs Non-Static: Describes variables ownership, not thread-safety. Shared static variable still require synchronization on atomic operations.

Unsynchronized Experiment: Increased concurrency causes lost updates, though percentage difference vary based on thread schedulling and hardware.

Thread-Safe Experiment: Serves as the control case where both totals mach and the percentage difference remains 0%.

IT-2403B

Answer to the Question

1. Answer: A static variable belongs to the class and is shared by all instances, whereas a non-static variable belongs to a specific object instance.

2. Answer: Because static variables are stored at the class level, making them accessible and shared across all instances and threads.

3. Answer: Because non-static variables are tied to individuals object instances created or used seperately by each thread.

4. Answer: To ensure the main thread waits for all worker threads to finish execution, before calculating and printing and final count.

5. Answer: Due to race condition and concurrent write conflicts resulting in lost updates.

6. Answer: No; The percentage difference is not guaranteed to increase motionically with threa count.

7. Answer: Due to non-deterministic nature of O thread scheduling and timing variations betw executions.

CamScanner

IT-24035

8. Answer: It ensures thread-safe, atomic operat
at the hardware level.

9. Answer: By passing a single, shared object
instance to all thread.

CamScanner